

SDL Practices

exercises

Henrik Kramselund
hlk@zencurity.com

October 4, 2026



Note: exercises marked with **▲** are considered important. These contain subjects that are essential for the course. Even if you don't work through the exercise, you might want to know the subjects covered by these.

Exercises marked with **❗** are considered optional. These contain subjects that are related to the course, but less important. You may want to browse these and if interested work through them. They may require more time than we have available during the course.

Contents

1	 Prepare your workstation 15 min	2
2	 Git tutorials - 15min	3
3	 Install and Run Kubernetes 45 min	5
4	 Run OWASP Juice Shop 45 min	8
5	 Setup JuiceShop and proxy - up to 20min	11
6	 CIS benchmark scan with kube-bench up to 30 min	13
7	 Trivy scan of container images in k3s up to 40 min	14
8	 JuiceShop Attacks 60min	16
9	 Truncate and Encoding Attacks JuiceShop up to 40min	18
10	 JuiceShop Login 15 min	20
11	 Git hook 30 min	21
12	 Small programs with data types 15min	23
13	 Pointers and Structure padding 30min	24
14	 Trying PMD static analysis 30 min	25

Preface

This material is prepared for use in SDL Practices course and was prepared by Henrik Kramselund, <https://www.zencurity.com>. It describes the networking setup and applications for trainings and courses where hands-on exercises are needed.

Further presentations are used which are available as PDF from <https://codeberg.org/kramse>
Look for SDL-practices-exercises in the repo security-courses.

These exercises are expected to be performed in a training setting with network connected systems. The exercises use a number of tools which can be copied and reused after training. A lot is described about setting up your workstation in the repo

<https://codeberg.org/kramse/kramse-labs>

Prerequisites

This material expects that participants have a working knowledge of TCP/IP from a user perspective. Basic concepts such as web site addresses and email should be known as well as IP-addresses, ports and common protocols like HTTP/HTTPS.

Have fun and learn

Exercise content

Most exercises follow the same procedure and have the following content:

- **Objective:** What is the exercise about, the objective
- **Purpose:** What is to be the expected outcome and goal of doing this exercise
- **Suggested method:** suggest a way to get started
- **Hints:** one or more hints and tips or even description how to do the actual exercises
- **Solution:** one possible solution is specified
- **Discussion:** Further things to note about the exercises, things to remember and discuss

Please note that the method and contents are similar to real-life scenarios and do not detail every step of doing the exercises. Entering commands directly from a book only teaches typing, while the exercises are designed to help you become able to learn and actually research solutions.

Exercise 1

Prepare your workstation 15 min

Objective:

Get your workstation ready for the hacker lab.

Purpose:

We will be running multiple tools

Suggested method:

- Create a folder for files from this course

Hints:

There are Git repositories on <https://codeberg.org/kramse/>

Solution:

When you have created a folder for files you are done, the rest can wait a bit.

Discussion:

Do we need to run local VMs, can we run Docker or Kubernetes instead – yes for some exercises, no for others.

You can work together in groups and only one workstation might have applications running.

Exercise 2

⚠ Git tutorials - 15min



Objective:

Try the program Git locally on your workstation

Purpose:

Running Git will allow you to clone repositories from others easily. This is a great way to get new software packages, and share your own.

Git is the name of the tool, and Codeberg and GitHub are popular sites for hosting git repositories.

Suggested method:

Run the program from your Linux VM. You can also clone from your Windows or Mac OS X computer. Multiple graphical front-end programs exist too.

First make sure your system is updated, as root run:

```
sudo apt update && sudo apt -y upgrade
```

You should reboot if the kernel is upgraded :-)

Second make sure your system has Git, ansible and my playbooks: (as root run, or with sudo as shown)

```
sudo apt -y install ansible git
```

Most important are Git clone and pull:

```
user@Projects:tt$ git clone https://codeberg.org/kramse/kramse-labs.git
Cloning into 'kramse-labs'...
remote: Enumerating objects: 283, done.
remote: Total 283 (delta 0), reused 0 (delta 0), pack-reused 283
Receiving objects: 100% (283/283), 215.04 KiB | 898.00 KiB/s, done.
Resolving deltas: 100% (145/145), done.

user@Projects:tt$ cd kramse-labs/

user@Projects:kramse-labs$ ls
LICENSE README.md core-net-lab lab-network suricatazeek work-station
user@Projects:kramse-labs$ git pull
Already up to date.
user@Projects:kramse-labs$ ls
core-net-lab/ kubernetes/ suricatazeek/ work-station/ README.md
docker-install/ lab-network/ web-security/ LICENSE
```

If you want to run Ansible playbooks, then run it with:

```
ansible-playbook -v example.yml
```

Hints:

Browse the Git tutorials on <https://git-scm.com/docs/gittutorial> and <https://guides.codeberg.org/activities/hello-world/>

We will not do the whole tutorials within 15 minutes, but get an idea of the command line, and see examples. Refer back to these tutorials when needed or do them at home.

Note: you don't need an account on Codeberg or Github to download/clone repositories, but having an account allows you to save repositories yourself and is recommended.

Solution:

When you have tried the tool and seen the tutorials you are done.

Discussion:

Before Git there have been a range of version control systems, see https://en.wikipedia.org/wiki/Version_control for more details.

Exercise 3

⚠ Install and Run Kubernetes 45 min

This is a fairly large exercise, and part of this course

Objective:

We will use containers, images, Linux for many exercises.

Kubernetes is an easy and standardized way to run many applications, which will allow us to deploy applications for testing quickly with just a few commands.

Purpose:

The course standardizes on Kubernetes k3s, so that everyone ends up on the same Kubernetes distribution and the exercise manifests should work the same for everyone.

Run different applications across your computers in a common way, to see the same versions and setups with minimal differences.

Suggested method:

This exercise has two install paths and one shared check at the end.

We expect a Kubernetes install for each team, up to three members per team.

Read this first:

If you already have Docker, Podman, VirtualBox, VMware, Hyper-V, WSL2 distros, KVM/libvirt, or any other container or virtualization tooling installed and want to keep it skip straight to Advanced Install below. Don't install Rancher Desktop on top of an existing setup; go to Advanced Install instead.

If your laptop is clean no containers, no VMs, nothing like that use Quick Start which uses Rancher Desktop.

Requirements are not especially high for Rancher Desktop, mostly you need:

64-bit CPU with hardware virtualization (Intel VT-x or AMD-V), enabled in BIOS/UEFI.

Mac OS X laptops require macOS 13 (Ventura) or newer. Apple Silicon (aarch64) or Intel (x86_64) CPU with VT-x. Recommended: 8 GB RAM, 4 CPU cores.

Quick Start (nothing installed yet)

This is the expected path for most of you.

1. **Uninstall leftovers first, if any.** If you've ever installed Docker Desktop, VirtualBox, or WSL distros for another course and aren't using them anymore, remove them before continuing running two virtualization stacks side by side is a common source of very confusing errors.
2. **Install Rancher Desktop** <https://rancherdesktop.io> . One installer per OS (Windows / macOS / Linux), free and open source, no license or account needed.
3. On first launch - use the following settings: **Enable Kubernetes:** on (is probably on by default) and **Engine:** containerd (select this, probably docker is default)
4. Wait for Rancher Desktop to show the cluster as running (first launch takes a few minutes it's downloading k3s).
5. Continue to Check Your Environment below.

Advanced Install (you already have something)

This path is for people who already have working tooling and don't want Rancher Desktop's VM layer on top of it. It's higher-level than Quick Start - you're expected to be comfortable following official docs for your specific tool.

Already have Docker / Docker Engine installed and running?

Install k3d <https://k3d.io> (k3s running inside Docker containers you already have). No new VM, reuses your existing Docker install.

```
k3d cluster create softsec
```

On Linux, with KVM/libvirt available (or no virtualization tooling at all)?

Install k3s <https://k3s.io> directly on the host. On Linux, Kubernetes doesn't need a VM at all the host is the container host.

```
curl -sfL https://get.k3s.io | sh -
```

On Windows or Mac, with WSL2 or another VM already set up that you want to keep using?

Install k3s inside that existing VM/WSL2 distro directly, following the k3s docs at <https://docs.k3s.io>, rather than adding Rancher Desktop's own VM alongside it.

Already running Docker Desktop's own Kubernetes, minikube, or kind? These work differently enough from k3s (different defaults, sometimes different Kubernetes version) that we'd still recommend adding k3s/k3d alongside for this course, so your environment matches the exercises. Keep your existing tool for anything else you use it for.

If your situation doesn't match any of the above, let's discuss.

Check Your Environment

Run once your cluster (Rancher Desktop or Advanced) is up.

1. Confirm kubectl can see your cluster:

```
kubectl get nodes
```

Expect one node, status Ready. Example output:

```
hlk@alice:~$ kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
lima-rancher-desktop Ready    control-plane  17m   v1.36.3+k3s1
```

2. Run a test pod:

```
kubectl run hello --image=nginx --restart=Never
kubectl get pods
```

Expect hello to reach Running within about a minute (`kubectl get pods -w` to watch it live). Example output:

```
hlk@alice:~$ kubectl run hello --image=nginx --restart=Never
// wait a bit and then
hlk@alice:~$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
hello     1/1     Running   0           5m45s
```

3. Confirm you can reach it:

```
kubectl port-forward pod/hello 8080:80
```

then open <http://localhost:8080> in a browser you should see the nginx welcome page. Ctrl+C to stop the port-forward. Example output:

```
hlk@alice:~$ kubectl port-forward pod/hello 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
Handling connection for 8080
Handling connection for 8080
```

4. Clean up:

```
kubectl delete pod hello
```

If all four steps worked, your environment is ready for the exercises. If step 1 fails, your cluster isn't running go back to whichever install path you used. If steps 2-4 fail but step 1 worked, flag it before the first exercise session.

Hints:

If you are low on memory and disk you might have problems.

Solution:

When you have a k8s/k3s cluster running as a team, then we are good.

Discussion:

Linux is free and everywhere. The tools we will run in this course are made for Unix, so they run great on Linux.

Exercise 4

i Run OWASP Juice Shop 45 min



Objective:

Let's try starting the OWASP Juice Shop

Purpose:

We will be doing some web hacking where you will be the hacker. There will be an application we try to hack, designed to optimise your learning.

It is named JuiceShop which is written in JavaScript

Suggested method:

Go to <https://github.com/juice-shop/juice-shop>

You need to have browsers and a proxy, plus a basic knowledge of HTTP.

Read the instructions for running juice-shop - **docker or Kubernetes is a simple way** and I often recommend that.

Running with Kubernetes follows.

First check your cluster: is it ready, and can you connect?

```
hlk@alice:~$ kubectl get node
NAME                STATUS    ROLES    AGE   VERSION
lima-rancher-desktop  Ready    control-plane  13d   v1.36.3+k3s1
hlk@alice:~$
```

Create a Deployment, name the file `juice-shop.yml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: juice-shop
  labels:
    app: juice-shop
spec:
  replicas: 1
  selector:
    matchLabels:
      app: juice-shop
  template:
    metadata:
      labels:
        app: juice-shop
    spec:
      containers:
        - name: juice-shop
          image: bkimminich/juice-shop:latest
          ports:
            - containerPort: 3000
---
apiVersion: v1
kind: Service
metadata:
  name: juice-shop
spec:
  selector:
    app: juice-shop
  ports:
    - port: 3000
      targetPort: 3000
  type: ClusterIP
```

This file can also be downloaded from Git `kramse-labs` in the directory `kubernetes`
`git clone https://codeberg.org/kramse/kramse-labs`

Run the deployment:

```
hlk@alice:kubernetes$ kubectl apply -f juice-shop.yml
deployment.apps/juice-shop created
service/juice-shop created
hlk@alice:kubernetes$
```

After a short while you should have a running pod/container and deployment:

```
h1k@alice:kubernetes$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
hello                0/1     Unknown   0           13d
juice-shop-86c67bcd4-1f87x  1/1     Running   0           2m27s
h1k@alice:kubernetes$ kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
juice-shop          1/1     1             1           2m38s
```

Wait for Running, then reach it using port forward:

```
h1k@laptop:~$ kubectl port-forward svc/juice-shop 3000:3000
```

Note: This needs to be done regularly. Familiarise yourself with `kubectl apply`, `kubectl delete` and `kubectl port-forward`

Check using: `http://127.0.0.1:3000`

Hints:

The application is very modern, very similar to real applications.

JuiceShop can be run as a Docker container, and sometimes running it on Kali is the easiest learning environment.

Solution:

When you have a running Juice Shop web application in your team, then we are good.

Discussion:

It has lots of security problems which can be used for learning hacking, and thereby how to secure your applications. It is related to the OWASP.org Open Worldwide Application Security Project which also has a lot of resources.

Sources:

<https://github.com/juice-shop/juice-shop>

<https://owasp.org/www-project-juice-shop/>

It is recommended to buy the Pwning OWASP Juice Shop Official companion guide to the OWASP Juice Shop from <https://leanpub.com/juice-shop> - suggested price USD 5.99

Exercise 5

i Setup JuiceShop and proxy - up to 20min

Objective:

Run JuiceShop with Burp proxy.

Start JuiceShop and make sure it works, visit using browser.

Then add a web proxy in-between. We will use Burp suite which is a commercial product, in the community edition.

Purpose:

We will learn more about web applications as they are a huge part of the applications used in enterprises and on the internet. Most mobile apps are also web applications in disguise.

By inserting a web proxy we can inspect the data being sent between browsers and the application.

The Burp proxy is an advanced tool! Don't be scared, we will use small parts at different times.

Suggested method:

You need to have browsers and a proxy, plus a basic knowledge of HTTP.

If you could install Firefox it would be great, and we will use the free version of Burp Suite, so please make sure you can run Java and download the free version from Portswigger from:

<https://portswigger.net/burp/communitydownload>

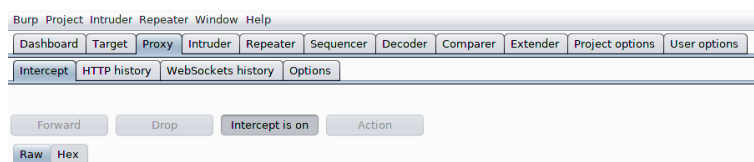
follow the Getting Started instructions at:

<https://support.portswigger.net/customer/portal/articles/1816883-getting-started-with-burp-suite>

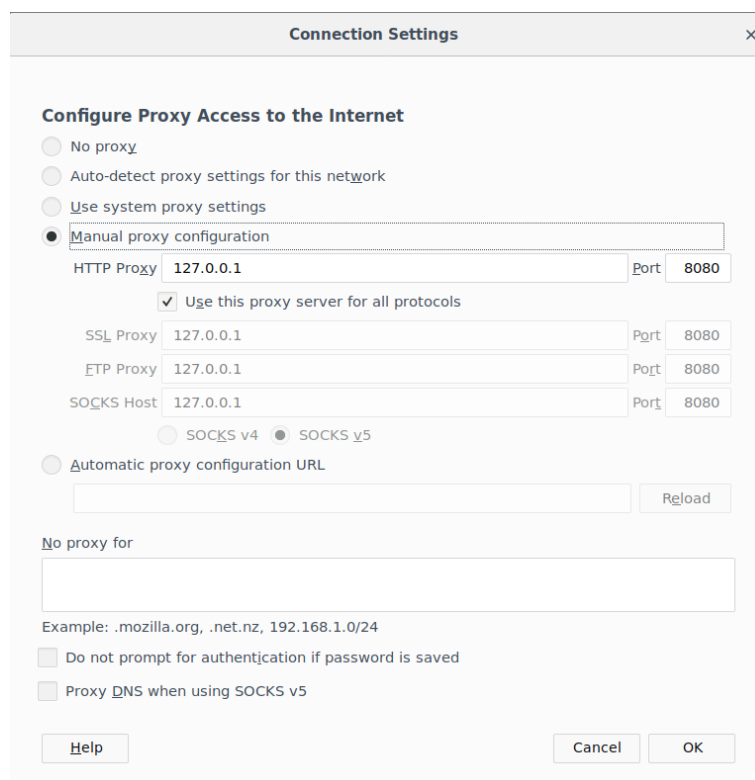
Hints:

Recommend running Burp on the default address and port 127.0.0.1 port 8080.

Note: Burp by default has **intercept is on** in the Proxy tab, press the button to allow data to flow.



Then setting it as proxy in Firefox:



After setting up proxy, you can visit `http://burp` and get a CA certificate that can be installed, making it easier to run against HTTPS sites.

The newest versions of Burp include a browser, making it much easier to run the tasks, pre-configured with proxy.

Firefox prevents hijacking localhost by default! Use `about:config` and set flag

`network.proxy.allow_hijacking_localhost` set to TRUE

Solution:

When web sites and servers start popping up in the Target tab, showing the requests and responses - you are done.

Your browser will alert you when visiting TLS enabled sites, HTTPS certificates do not match, as Burp is doing a person-in-the-middle. You need to select advanced and allow this to continue.

Discussion:

Since Burp is often updated I use a small script for starting Burp which I save in `~/bin/burp` – don't forget to add to PATH and `chmod +x bin/burp`.

```
#!/bin/sh
DIRNAME=`dirname $0`
BURP=`ls -ltra $DIRNAME/burp*.jar | tail -1`
java -jar -Xmx6g $BURP &
```

When running in production testing real sites, I typically increase the memory available using JDK / Java settings like `-Xmx16g`

Exercise 6

i CIS benchmark scan with kube-bench up to 30 min

Objective:

We will run kube-bench against our local k3s cluster to check it against the CIS Kubernetes Benchmark – a checklist of security configuration recommendations for Kubernetes clusters.

Purpose:

See what a configuration/hardening scanner looks like, as opposed to a vulnerability scanner like Trivy from another exercise. kube-bench doesn't look for known CVEs – it checks whether the cluster itself (API server, kubelet, etcd, etc.) is configured according to security best practice.

Suggested method:

Run kube-bench as a Job inside your k3s cluster using `kubect1`, since that gives it access to the node's config files and binaries needed to check things like file permissions. Then read through the output.

Hints:

The kube-bench project publishes ready-made Kubernetes manifests to run it as a Job:

```
hlk@laptop:~$ kubect1 apply -f https://raw.githubusercontent.com/aquasecurity/kube-bench/main/job.yaml
```

Wait for it to finish, then read the logs:

```
hlk@laptop:~$ kubect1 get pods | grep kube-bench
hlk@laptop:~$ kubect1 logs job/kube-bench
```

k3s bundles components a bit differently than a “vanilla” kubeadm cluster (e.g. no separate etcd pod, different config file paths), so don't be surprised if some checks come back as [INFO] or [WARN] “not applicable” rather than a clean pass/fail – there is a k3s-specific config/target you can look for in the kube-bench docs if you want more accurate results. Clean up afterwards:

```
hlk@laptop:~$ kubect1 delete -f https://raw.githubusercontent.com/aquasecurity/kube-bench/main/job.yaml
```

Solution:

You are done once you can point to at least one [FAIL] or [WARN] result in the output, explain in your own words what the check was actually verifying, and say whether it's something you'd realistically fix on a student laptop cluster or only on a production cluster.

Discussion:

Look up the Trivy scanner, and compare kube-bench to Trivy: Trivy tells you what known-vulnerable software is running; kube-bench tells you whether the cluster is configured safely, regardless of software versions. Both are needed. Discuss:

- Why a perfectly patched cluster can still be insecure if it's misconfigured (e.g. anonymous API access enabled)
- Why some checks are unrealistic to fully satisfy on a local dev cluster (e.g. those about running on dedicated, hardened hosts) but matter a lot in production
- How this fits into a broader “Secure by Design” pipeline: SCA (Trivy) + config auditing (kube-bench) + your own code's static analysis (PMD)

Exercise 7

📌 Trivy scan of container images in k3s up to 40 min

Objective:

We will use Trivy to scan the container images actually running in our local k3s cluster (the one set up with Rancher Desktop earlier in the course) for known vulnerabilities. We use `kubectl` to find out which images are running, and Trivy to scan them.

Purpose:

Get hands-on experience with a real vulnerability scanner and see how it maps found issues back to:

- CVE identifiers
- CWE categories (compare with the OWASP Top 10 and CWE Top 25 discussed in class)
- Severity ratings (CRITICAL / HIGH / MEDIUM / LOW)

This is your first taste of Software Composition Analysis (SCA) – scanning not code you wrote, but the third-party layers you depend on (base images, OS packages, libraries).

Suggested method:

Make sure your k3s cluster (Rancher Desktop) is running and that you have something deployed in it – e.g. NGINX from an earlier exercise. If nothing is deployed yet, deploy something first. Install Trivy on your workstation (not inside the cluster). Follow the instructions from <https://trivy.dev/docs/latest/getting-started/installation/>

Use `kubectl` to list which images are actually running, then scan those images directly with Trivy.

Hints:

Find the images currently running in your cluster:

```
hlk@laptop:~$ kubectl get pods -A -o jsonpath="{.items[*].spec.containers[*].image}" | tr -s ' ' '\n' | sort -u
```

Scan a single image:

```
hlk@laptop:~$ trivy image bkimminich/juice-shop:latest
```

Trivy has a dedicated Kubernetes mode that talks to your cluster directly via your kubeconfig, and will enumerate and scan all running workloads for you – try it once you have scanned a single image manually:

```
hlk@laptop:~$ trivy k8s --report summary cluster
```

If you want vulnerability data only (skip secrets/misconfig scanning, faster):

```
hlk@laptop:~$ trivy image --scanners vuln bkimminich/juice-shop:latest
```

Try filtering to only show what matters for patching:

```
hlk@laptop:~$ trivy image --severity CRITICAL,HIGH bkimminich/juice-shop:latest
```

Solution:

A finished run produces a table (or JSON, with `-f json`) listing, for each vulnerable package, the installed version, the fixed version (if one exists), the CVE ID, and severity. You are done once you can:

- Explain what one specific CVE in the output actually is (look it up)
- State whether a fix is available for it
- Say whether it applies to the OS layer (e.g. Debian/Alpine packages) or the application layer (e.g. npm packages)

Discussion:

Trivy will usually report far more vulnerabilities than you can realistically fix – this is normal, and mirrors real-world alert fatigue in vulnerability management. Discuss:

- Why base image choice (e.g. **alpine** vs **debian** vs **distroless**) drastically changes the vulnerability count
- Why “no fix available” is common and what you’d do about it in practice (accept risk, find an alternative package, mitigate elsewhere)
- How this connects to CI/CD – scanning should happen before deploy, not just ad-hoc like we just did
- The difference between scanning an image (Trivy) and scanning your own source code for weaknesses (static analysis, e.g. PMD from an earlier exercise) – Trivy tells you about vulnerabilities in dependencies, not bugs in your own code

Exercise 8

⚠ JuiceShop Attacks 60min



Objective:

Hack a web application!

Try a few attacks in the JuiceShop with web proxy

The OWASP Juice Shop is a pure web application implemented in JavaScript. In the frontend the popular AngularJS framework is used to create a so-called Single Page Application. The user interface layout is provided by Twitter's Bootstrap framework - which works nicely in combination with AngularJS. JavaScript is also used in the backend as the exclusive programming language: An Express application hosted in a Node.js server delivers the client-side code to the browser. It also provides the necessary backend functionality to the client via a RESTful API.

...

The vulnerabilities found in the OWASP Juice Shop are categorized into several different classes. Most of them cover different risk or vulnerability types from well-known lists or documents, such as OWASP Top 10 or MITRE's Common Weakness Enumeration. The following table presents a mapping of the Juice Shop's categories to OWASP and CWE (without claiming to be complete).

Category Mappings

Category	OWASP	CWE
Injection	A1:2017	CWE-74
Broken Authentication	A2:2017	CWE-287, CWE-352
Forgotten Content	OTG-CONFIG-004	
Roll your own Security	A10:2017	CWE-326, CWE-601
Sensitive Data Exposure	A3:2017	CWE-200, CWE-327, CWE-328, CWE-548
XML External Entities (XXE)	A4:2017	CWE-611
Improper Input Validation	ASVS V5	CWE-20
Broken Access Control	A5:2017	CWE-22, CWE-285, CWE-639
Security Misconfiguration	A6:2017	CWE-209
Cross Site Scripting (XSS)	A7:2017	CWE-79
Insecure Deserialization	A8:2017	CWE-502
Vulnerable Components	A9:2017	
Security through Obscurity		CWE-656

Source: Pwning OWASP Juice Shop

Purpose:

Try out some of the described web application flaws in a controlled environment. See how an attacker would be able to gather information and attack through HTTP, browser and proxies.

Suggested method:

Start the web application, start Burp or another proxy - start your browser.

Access the web application through your browser and get a feel for how it works. First step is to register your user, before you can shop.

Don't forget to use web developer tools like the JavaScript console!

Then afterwards find and try to exploit vulnerabilities, using the book from Björn and starting with some easy ones:

Suggested list of starting vulns:

- Admin Section Access the Admin Section
- Error handling Provoke an error
- Forged Feedback Post some feedback in another user's name.
- Access a confidential document
- Forgotten Sales Backup Access a salesman's forgotten backup file.
- Retrieve a list of all user credentials via SQL Injection

Hints:

The complete guide Pwning OWASP Juice Shop written by Björn Kimminich is available as PDF which you can buy, or you can read it online at:

<https://pwning.owasp-juice.shop/companion-guide/latest/>

Solution:

You decide for how long you want to play with JuiceShop.

Do know that some attackers on the internet spend all their time researching, exploiting and abusing web applications.

Discussion:

The vulnerabilities contained in systems like JuiceShop mimic real ones, and do a very good job. You might not think this is possible in real applications, but there is evidence to the contrary.

Using an app like JS instead of real applications with flaws allows you to spend less on installing apps, and more on exploiting.

Exercise 9

⚠ Truncate and Encoding Attacks JuiceShop up to 40min

Objective:

Try out some of the problems described in the Juice Shop book using active methods.

Purpose:

The book describes problems with encodings but it can feel a bit fluffy unless you try and see for yourself. We have the JuiceShop which has errors similar to these.

Suggested method:

There is an error in the JuiceShop that can be abused for reading files using encoding %2500.

Try to download the file <http://localhost:3000/ftp/eastere.gg>

It should be possible to even retrieve the content of a file like `C:\Windows\system.ini` or `/etc/passwd` from the server and see if you can read a file. If they exist. This is related to XXE attacks, and seems hard to get working.

Spoiler alert next page!

Hints:

It's OK to use the solution and work through the example. `http://localhost:3000/ftp/easteregg%2500.md`

Also make sure to use `NODE_ENV=unsafe` flag when running docker if you want to try the XXE vuln!

```
export NODE_ENV=unsafe
docker run --rm -p 0.0.0.0:3000:3000 bkimminich/juice-shop
```

Solution:

When you feel you understand the problem of encoding/decoding and sending XML files to an application, reading files, you are done.

Discussion:

Another problem is the filtering done in applications.

In the JuiceShop we can access using URLs like this on the About Us page:

```
http://localhost:3000/ftp/legal.md?md_debug=true
```

Consider if the URL would match on `.md` and we were able to send a large filename ending in `loongfilename.md`, but when truncated cut off exactly the `.md` part so we referenced another file.

Exercise 10

⚠ JuiceShop Login 15 min

```
models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email} || ''}'  
AND password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`,  
{ model: models.User, plain: true })
```

Objective:

Try to find the JuiceShop login box implementation, we know it is vulnerable to SQL injection.

Purpose:

Seeing bad code is a design pattern – anti-pattern.

Suggested method:

Find the source code, look for the database lookups.

Hints:

The JuiceShop software is open source, and available at GitHub:

<https://github.com/juice-shop/juice-shop>

Git clone and searching locally might give the best results.

In this case we can search for `SELECT * FROM`, using a simple tool like `grep`:

```
user@Projects:juice-shop$ grep -ril SELECT | egrep -v "test|frontend|static"  
...  
REFERENCES.md  
routes/vulnCodeFixes.ts  
routes/search.ts  
routes/login.ts // oohhhh looks interesting  
routes/countryMapping.ts  
routes/vulnCodeSnippet.ts  
config/oss.yml  
config/mozilla.yml  
config/default.yml  
README.md
```

Solution:

When you have found examples of the database lookups, you are done. See also discussion below though.

Discussion:

Think about how this could be changed. How much would it require to change this into prepared statements? Also, having good source code tools helps a lot! Finding problems, getting an overview of code etc.

Exercise 11

⚠ Git hook 30 min

Objective:

Try using a Git hook locally on your workstation, to prevent something.

Purpose:

Running Git with hooks will allow you to perform actions when adding source code. For security we can prevent you from adding something to the source tree which breaks policies we agreed.

First read the documentation:

<https://git-scm.com/book/en/v2/Customizing-Git-Git-Hooks>

Note: today we only use client side hooks, for better security we should add hooks on the server side.

Using our existing repository:

```
user@Projects:projects$ git clone https://codeberg.org/kramse/kramse-labs.git
Cloning into 'kramse-labs'...
user@Projects:projects$ cd kramse-labs/
user@Projects:kramse-labs$ cd .git/hooks
user@Projects:kramse-labs/.git/hooks$ cat pre-commit.sample
// Look at this file, and activate it using:
user@Projects:kramse-labs/.git/hooks$ cp pre-commit.sample pre-commit
```

Back in the repository try adding a new file with bad name:

```
user@Projects:kramse-labs$
user@Projects:kramse-labs$ touch henrikEØÅ
user@Projects:kramse-labs$ git add henrikEØÅ
user@Projects:kramse-labs$ git commit -m "Adding henrikEØÅ"
Error: Attempt to add a non-ASCII file name.
This can cause problems if you want to work with people on other platforms.
To be portable it is advisable to rename the file.
```

If you know what you are doing you can disable this check using:

```
git config hooks.allow
```

Suggested method:

Run the program Git from your Debian Linux VM

Hints:

Many hooks will depend on the language and what your policy states. Some hooks will be easy to implement, others will require others to agree. Think tabs vs spaces which will forever stay unresolved.

Solution:

When you have tried adding a file with a bad name, and gotten an error, you are done. Feel free to experiment more with the hook.

Discussion:

Many examples can be found on the internet.

Example links:

- <https://blogs.vmware.com/opensource/2020/02/03/git-repo-pre-commit-hooks/> which links to the next one
- A framework for managing and maintaining multi-language pre-commit hooks.
<https://pre-commit.com/>
- Blog showing how to use this framework for checking a Kubernetes file:
<https://thechief.io/c/hebaeid/how-to-start-left-with-security-using-git-pre-commit-hooks>

Exercise 12

⚠ Small programs with data types 15min

Objective:

Try out small programs similar to:

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv)
{
    (void) argc; (void) argv;
    short int i1 = 32767;
    printf("First debug int is %d\n", i1);
    i1++;
    printf("Second debug int is now %d \n", i1);
}
```

```
user@Projects:programs$ gcc -o int1 int1.c && ./int1
```

```
First debug int is 32767
```

```
Second debug int is now -32768
```

Purpose:

See actual overflows when going above the maximum for the selected types.

Suggested method:

Compile program as is. Run it. See the problem.

Then try changing the int type, try with signed and unsigned. Note differences

Hints:

Use a calculator to find the maximum, like 2^{16} , 2^{32} etc.

Solution:

When you have tried adding one to a value and seeing it going negative, you are done.

Discussion:

Exercise 13

⚠️ Pointers and Structure padding 30min

Objective:

Look at some real code from Suricata and Zeek, note how they prevent structure padding.

Purpose:

These applications, usually used for security, dissect raw packets which cannot be trusted.

Suggested method:

Download the source for some software - either of:

- Zeek from <https://zeek.org/get-zeek/>
- Suricata from <https://www.openinfosecfoundation.org/download/>

Unpack using `tar xzf` and use an editor to look up DNS or other packets.

Hints:

DNS is a complex protocol, but looking at the header files should give you an idea. Try going into `src` and doing `less *dns*.h` or use an editor.

Solution:

When you have seen the code for a few `struct` you are done.

Notice structs with `__attribute__((__packed__))`. Note: This ensures that structure fields align on one-byte boundaries - on all architectures.

Maybe also investigate the rest of the file `decode-vxlan.c` if you downloaded Suricata.

Discussion:

The manual for the GNU Compiler Collection (GCC) can be found at:

<https://gcc.gnu.org/onlinedocs/gcc-5.2.0/gcc/Type-Attributes.html>

packed

This attribute, attached to struct or union type definition, specifies that each member (other than zero-width bit-fields) of the structure or union is placed to minimize the memory required. When attached to an enum definition, it indicates that the smallest integral type should be used.

Bonus, can we find some structs missing this?

Exercise 14

⚠ Trying PMD static analysis 30 min

Objective:

Try the program PMD locally on your workstation

Purpose:

Running PMD will allow you to use static analysis for code.

Suggested method:

Run the program from your Debian Linux VM, this tool is free and easy to get running. It uses Java, so if you like run it on your Windows or Mac instead.

Follow instructions from the Getting Started

https://pmd.github.io/latest/pmd_userdocs_installation.html

The download is from latest, so check the releases page: <https://github.com/pmd/pmd/releases>

```
$ sudo apt install openjdk-17-jre
$ cd $HOME
$ wget https://github.com/pmd/pmd/releases/download/pmd_releases/6.49.0/pmd-bin-6.49.0.zip
$ unzip pmd-bin-6.49.0.zip
$ alias pmd="$HOME/pmd-bin-6.49.0/bin/run.sh pmd"
$
```

Note: this only creates the alias `pmd` for this session. To make this more permanent, you could add this to a profile like `.bashrc`

Next get some source code and run PMD:

```
$ git clone --branch rel/2.17.2 https://gitbox.apache.org/repos/asf/logging-log4j2.git
... downloads the source code for log4j
$ pmd -d logging-log4j2 -R rulesets/java/quickstart.xml -f text
```

Hints:

PMD uses Java, so there should be a JDK/JRE on the system, I install the one from OpenJDK above.

You may need to adjust the version numbers for the JDK/JRE, PMD and select another version of log4j to download.

Solution:

When you have gotten a run of PMD going, you are done.

Discussion:

Doing the above probably outputs more than 4500 lines from the PMD program!

How would you proceed?

Some are tedious but easy to fix, like `Unused import` – importing some library which is not really used. Things like `empty method`, `empty catch block` etc. may be source missing.

First time use of a new tool will probably find a LOT.

If you are using Maven you could also use their reporting

<https://maven.apache.org/plugins/maven-pmd-plugin/project-reports.html>